

Istruzioni

Dario Pescini

Università degli Studi di Milano-Bicocca

Dipartimento di Statistica e Metodi Quantitativi

dario.pescini@unimib.it

Elementi di un programma

 variabili

 istruzioni

 strutture di controllo

Elementi di un programma

 variabili

 istruzioni

 strutture di controllo

Esecuzione

Esistono diversi tipi di istruzioni o operatori:

- assegnamento
- matematici, incremento/decremento
- confronto, logici
- I/O
- composti o espressioni
- per il controllo del flusso delle informazioni

≡ operatori di assegnamento

Come indicato dal nome, servono ad **assegnare un valore ad una variabile**

```
a = 5
```

Assegnamento: =

- **a** variabile a cui assegnare il valore
- **=** operatore di assegnamento
- **5** valore da assegnare alla variabile

≡ operatori aritmetici

- + somma
- sottrazione
- * prodotto
- ** elevamento a potenza
- / divisione
- // divisione tra interi
- % resto

le due divisioni: / e //

In Python 3 / e // sono **due operatori diversi**, non due modi di scrivere lo stesso:

	/	//
3 / 2	1.5	1
3 / 2.0	1.5	1.0
5 / 2	2.5	2

la regola, in una riga

/ restituisce **sempre un float**, anche fra due interi. // tronca verso il basso e conserva il tipo degli operandi.

divisione - Esercizio

Scrivere un programma che prenda in input 2 numeri interi, a e b, e restituisca a video

- quoziente a / b
- resto $a \% b$
- prodotto del quoziente per b

divisione - Esercizio

Scrivere un programma che prenda in input 2 numeri interi, a e b, e restituisca a video

- quoziente a / b
- resto $a \% b$
- prodotto del quoziente per b

```
a = 257
b = 11

quoziente = a // b
resto = a % b
prodotto = b * quoziente

print("a = %d b = %d" % (a, b))
print("a / b = %d con resto %d" % (quoziente, resto))
print("b * quoziente = ", prodotto)
print("prodotto + resto = ", prodotto + resto)
```

divisione - Esercizio

Scrivere un programma che prenda in input 2 numeri reali, a e b, e restituisca a video

- quoziente a / b
- resto $a \% b$
- prodotto del quoziente per b

divisione - Esercizio

Scrivere un programma che prenda in input 2 numeri reali, a e b, e restituisca a video

- quoziente a / b
- resto $a \% b$
- prodotto del quoziente per b

```
a = 257.0
b = 11.0

quoziente = a / b
resto = a % b
prodotto = b * quoziente

print("a = %f b = %f" % (a, b))
print("a / b = %f con resto %f" % (quoziente, resto))
print("b * quoziente = ", prodotto)
print("prodotto + resto = ", prodotto + resto)
```

≡ stringhe: +, *

concatenazione: +

L'operatore di concatenazione **+** unisce due stringhe.

stringaA **+** stringaB

```
>>> stringA = 'Hello '  
>>> stringB = 'World !'  
>>> print(stringA + stringB)  
Hello World !
```

ripetizione: *

L'operatore di ripetizione ***** replica più volte la stringa.

numero_volte ***** stringa

```
>>> print(3 * stringA)  
Hello Hello Hello
```

≡ operatori incremento/decremento

a += valore

incremento: +=

- a variabile da incrementare
- += incrementa di **valore** la variabile **a**

≡ operatori incremento/decremento

`a += valore`

incremento: +=

- `a` variabile da incrementare
- `+=` incrementa di `valore` la variabile `a`

```
>>> a = 0
>>> a += 5
>>> print(a)
5
```

≡ operatori incremento/decremento

`a -= valore`

decremento: `-=`

- `a` variabile da decrementare
- `-=` decrementa di `valore` la variabile `a`

≡ operatori incremento/decremento

`a -= valore`

decremento: `-=`

- `a` variabile da decrementare
- `-=` decrementa di `valore` la variabile `a`

```
>>> a = 0
>>> a -= 5
>>> print(a)
-5
```


≡ operatori di confronto

- > maggiore
- >= maggiore o uguale
- < minore
- <= minore o uguale
- == uguaglianza
- != disuguaglianza
- is identità
- is not non identità

Restituiscono **True** **False** a seconda che il confronto sia **vero falso**.

```
>>> a = 5
>>> b = 10
>>> a > b
False
```

≡ operatori di confronto

É possibile immagazzinare l'informazione {Vero, Falso} in una variabile di tipo "Booleano" ∈ {True, False}

```
>>> a = True
>>> type(a)
<type 'bool'>
>>> _
```

```
>>> a = True
>>> b = False
>>> c = a == b
```

≡ operatori di appartenenza

Verificano se un elemento appartiene ad un gruppo:

`in` appartiene

`not in` non appartiene

Restituiscono `True` `False` a seconda che il test sia `vero` `falso`.

```
>>> s = 'appartenenza'
>>> print('z' in s)
True
>>> print('b' in s)
False
```

≡ operatori logici

Ricevono in **ingresso** variabili di tipo “Booleano” e restituiscono **un valore** ancora Booleano:

$$F : \{\text{True, False}\}^n \rightarrow \{\text{True, False}\}$$

operatori logici

Ricevono in **ingresso** variabili di tipo “Booleano” e restituiscono **un valore** ancora Booleano:

not: $\{\text{True}, \text{False}\} \rightarrow \{\text{True}, \text{False}\}$

not

A	not(A)
V	F
F	V

value = not A

operatori logici

Ricevono in **ingresso** variabili di tipo “Booleano” e restituiscono **un valore** ancora Booleano:

and: $\{\text{True}, \text{False}\}^2 \rightarrow \{\text{True}, \text{False}\}$

and

	V	F
V	V	F
F	F	F

value = **A** and **B**

operatori logici

Ricevono in **ingresso** variabili di tipo “Booleano” e restituiscono **un valore** ancora Booleano:

or: $\{\text{True}, \text{False}\}^2 \rightarrow \{\text{True}, \text{False}\}$

or

	V	F
V	V	V
F	V	F

value = A **or** B

≡ operatori I/O: print

```
print(una_Stringa)
```

print - scrittura

- `print()` è l'istruzione per stampare una stringa a video
- `una_Stringa` è la stringa che verrà visualizzata. si può sostituire la variabile con una stringa racchiusa tra singoli ' ' o doppi " " apici.

```
print ("del testo %d altro testo %d" % (var1, var2) )
```

print - scrittura

- `"del testo %d altro testo %d"` è la stringa che verrà visualizzata dove %d è l'identificatore di un parametro.
- `% (var1, var2)` è l'elenco dei valori che saranno assegnati ai parametri

≡ operatori I/O - formattazione

Per inserire nelle stringhe dei valori associati a variabili si utilizzano degli identificatori di formattazione:

%d	sostituisci con un intero
%f	sostituisci con un float
%r	sostituisci con contenuto raw della variabile
%s	sostituisci con una stringa
\t	sostituisci con una tabulazione
\n	sostituisci con un a capo

≡ operatori I/O: print

```
>>> a = 10

>>> print("si possono" + "concatenare le stringhe e mescolare valori numerici", a)
si possonoconcatenare le stringhe e mescolare valori numerici 10

>>> print("pippo %d " % 10)
pippo 10

>>> print("pippo %d e %f" % (10, 10.0))
pippo 10 e 10.000000

>>> print("pippo %d e %r" % (10, 10.0))
pippo 10 e 10.0

>>> print("pippo %d e %s" % (10, 10.0))
pippo 10 e 10.0
```

≡ operatori I/O: input

```
una_variabile = input("inserisci un valore")
```

input - lettura

- `input( )` è l'istruzione per leggere un valore da terminale
- `"inserisci un valore"` è la stringa che viene mostrata all'utente
- `una_variabile` è la variabile a cui assegnare il **valore**
- `una_variabile` è una stringa

≡ operatori I/O: eval(input())

```
una_variabile = eval(input("inserisci un valore"))
```

eval - conversione numerica

- `eval( )` è l'istruzione per convertire una stringa in un valore (se possibile). Il tipo viene assegnato automaticamente.
- `"inserisci un valore"` è la stringa che viene mostrata all'utente
- `una_variabile` è la variabile a cui assegnare il **valore** inserito dall'utente

≡ convertire quello che si legge

`eval` indovina il tipo. **Dirlo esplicitamente** è la strada del corso:

```
eta = int(input("quanti anni hai? "))  
temperatura = float(input("temperatura in gradi: "))
```

perché è quella da usare

- `input()` restituisce **sempre** una stringa: "5" + "3" dà "53", non 8
- `int()` e `float()` dicono nel codice **che cosa ci si aspetta**, e se l'utente scrive altro l'errore arriva subito
- `eval()` esegue quello che riceve: su un valore battuto da altri è una porta aperta

≡ le f-string

Una stringa preceduta da **f**: fra graffe si scrive il **nome della variabile**, nel punto in cui il valore deve comparire.

```
citta = "Milano"  
giorno = 12  
tmax = 9.4  
  
print(f"Il {giorno} gennaio a {citta} la massima è stata {tmax} gradi")
```

Il 12 gennaio a Milano la massima è stata 9.4 gradi

Rispetto a %d e %f, il valore sta **dove lo si legge** e non in coda alla riga.

≡ f-string: quante cifre

Dopo i due punti si dice **come** stampare il valore:

```
>>> media = 44 / 7
>>> print(f"media: {media}")
media: 6.285714285714286

>>> print(f"media: {media:.2f}")
media: 6.29
```

- `:.2f` sono **due cifre decimali**: fa quello che faceva `%f`, scritto accanto al valore
- le due forme convivono, e nel corso **si possono usare entrambe**